ELTE EÖTVÖS LORÁND UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

2026. március 1.

Content in this Practice I

Informations

Contact information

- Name: Gábor Pusztai
- Files: Canvas and `wornox.web.elte.hu`
- Teams: Gábor Pusztai (AFBDPK)
- Email: `pusztaigabor@inf.elte.hu`
- Useful links:
  - `aszt.inf.elte.hu/~asvanyi/ds/AD.pdf`
  - Book: Introduction to Algorithms (Cormen-Leiserson-Rivest-Stein)
- *Special thanks to* ***Nóra Harrach*** *for the slides and help in this subject.*

Lesson information

- Personal attendance is mandatory: only 3 skips are allowed
- 2 exams during the class:
  - First test:
    - When: around the 7th week
    - Time and location: at this lesson
    - Format: on paper | Score: 50 points (minimum: 20)
  - Second test:
    - When: around the 13th week
    - Time and location: at this lesson
    - Format: on paper | Score: 50 points (minimum: 20)
- The final grade will be determined by the sum of the exams: 50+50=100. One rewrite opportunity at the end of the semester.
  (2) 40 - 54, (3) 55 - 69, (4) 70 - 84, (5) 85 - 100

Semester information

- Progress plan for the semester
  - 1, Introduction; Bubble sort; Selection sort
  - 2, Insertion sort; Stack; Polish notation
  - 3, List (one-way)
  - 4, List (two-way)
  - 5, Merge sort
  - 6, Quick sort; Queue
  - **7, First exam**
  - 8, Binary tree
  - 9, Binary search tree
  - 10, Heap; Priority queue; Heap sort
  - 11, Hash table
  - 12, Linear; Radix; Bucket; Counting sort
  - **13, Second exam**

Introduction

Importance of Algorithms

Why is it important to study Algorithms and data structures?

- To solve problems with finite computational resources:
  - time *("Time is money", "You can get back money after you spend it, but once the time is spent, you can never get it back.")*
  - memory *(nowadays inexpensive, but still not infinite)*
- Optimization:
  - time *(waiting time)*
  - memory *(e.g. cloud space is expensive)*

Algorithms

What is an algorithm?

- *An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output in a `finite amount of time`).*
- *An **algorithm** is thus a sequence of computational steps that transform the input into the output.*
- *You can also view an **algorithm** as a tool for solving a well-specified computational problem.*

(Cormen-Lieserson-Rivest-Stein - Introduction to algorithms - 1.1 Algorithms)

Time complexity

Time complexity: Reflects some abstract time. We count the subroutine calls + the number of loop iterations during the program's run.
Notations:

- $MT(n)$ (Maximum Time),
- $AT(n)$ (Average or expected Time)
- $mT(n)$ (minimum Time)

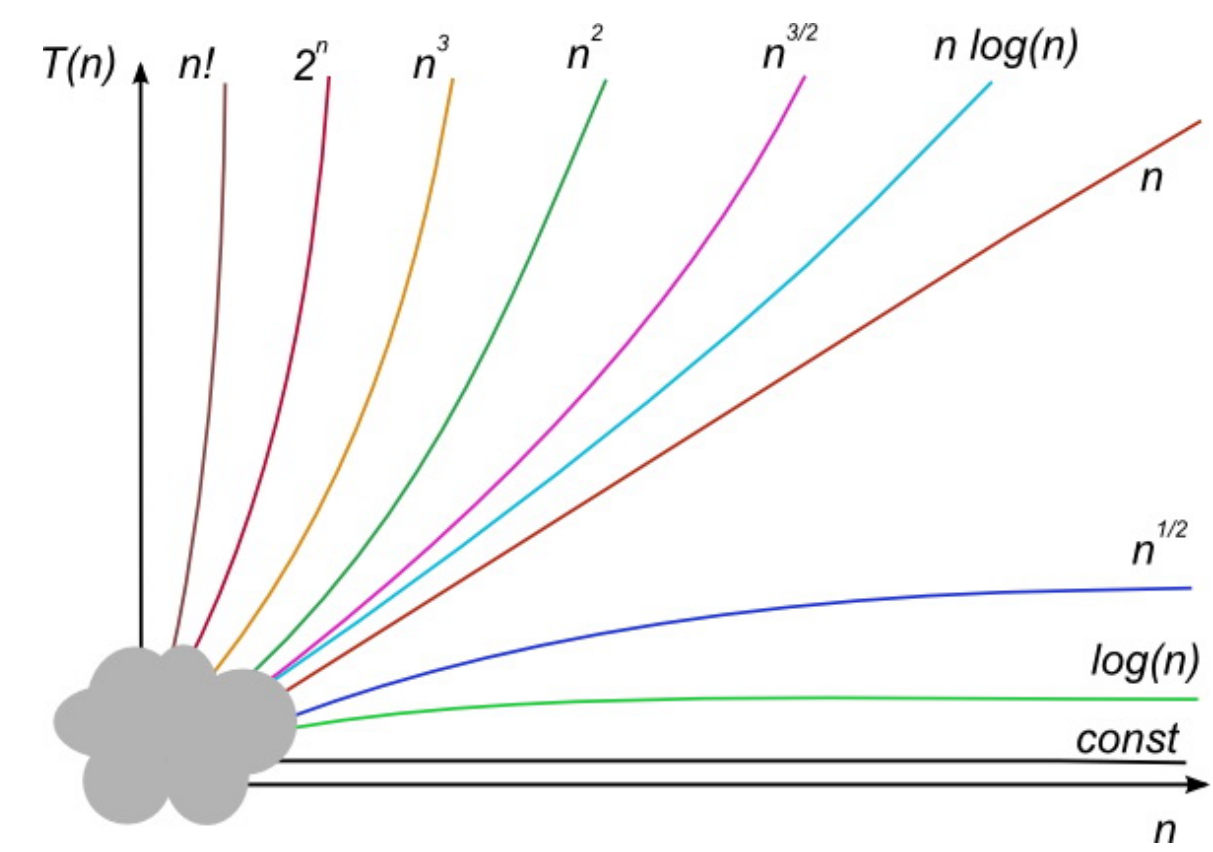
$MT(n) \geq AT(n) \geq mT(n)$
Typically, the time complexities of the algorithms are not calculated precisely. Only we calculate their *asymptotic order* or make *asymptotic estimation(s)*.

Asymptotic estimation(s)

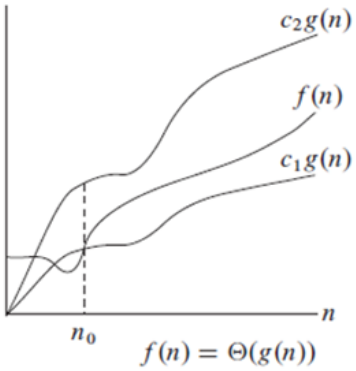
- Asymptotic UPPER estimate:
  - O (big-O, Omicron)
  - runtime is at *most** our given value
- Asymptotic LOWER estimate
  - Ω (Omega)
  - runtime is at *least** our given value
- Asymptotic UPPER AND LOWER estimate
  - Θ (Theta)
  - runtime is *"expected somewhere near"** our given value

**More precise definitions in Lecture*

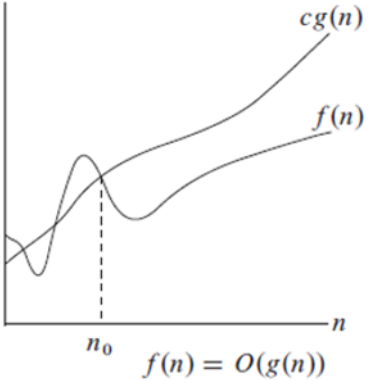
Asymptotic estimation(s)



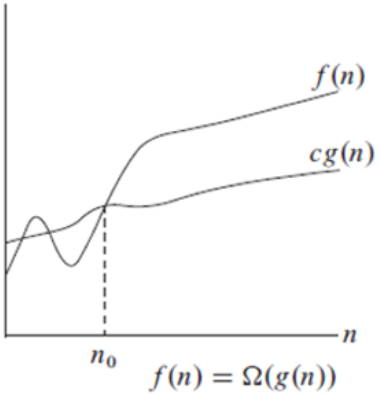
Asymptotic estimation(s)



$f(n) = \Theta(g(n))$



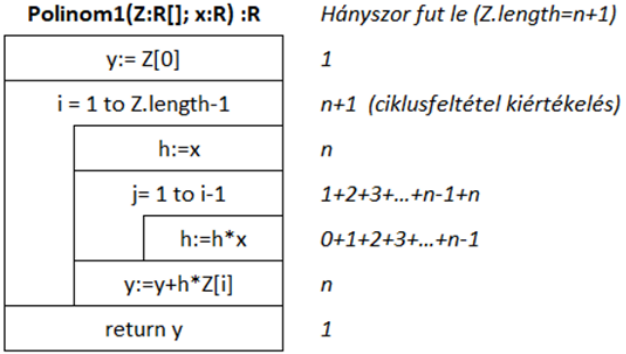
$f(n) = O(g(n))$



$f(n) = \Omega(g(n))$

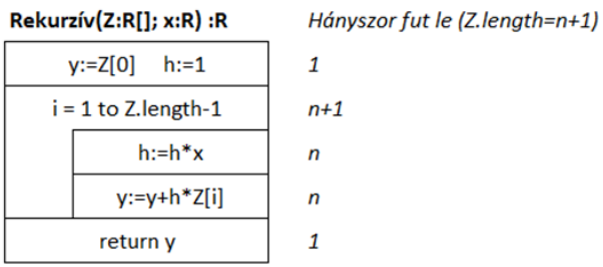
Computing the value of a polynomial

- Let us examine the number of loop iterations it(n), multiplications S(n), and additions Ö(n), as a function of the polynomial's degree.



$$S(n) = \frac{n * (n+1)}{2} = \frac{n^2 + n}{2}$$

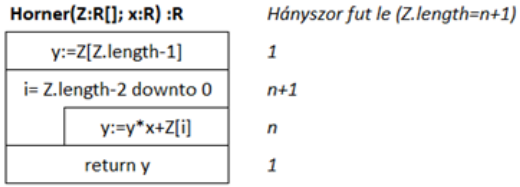
$Ö(n) = n$ *it(n) = S(n)*



$S(n) = 2 * n$ *it(n) = n*

$Ö(n) = n$

Computing the value of a polynomial



$S(n) = n$

$\ddot{O}(n) = n$

it(n) = n

	Polynomial	Recursive	Horner
S(n)	$\theta(n^2)$	$\theta(n)$	$\theta(n)$
Ö(n)	$\theta(n)$	$\theta(n)$	$\theta(n)$
it(n)	$\theta(n^2)$	$\theta(n)$	$\theta(n)$

Space complexity

Space complexity: Is an abstract measure of the algorithm's space (memory) requirements. It depends on the depth of the subroutine calls and the sizes of the data structures.
Notations:

- $MS(n)$ (Maximum Space complexity),
- $AS(n)$ (Average or expected Space complexity)
- $mS(n)$ (minimum Space complexity)

$MS(n) \geq AS(n) \geq mS(n)$

Data Structures and Types

Data Structures and Data Types

Data structure:

- a way to store data and
- organize data.

Why use different Data Structures?

- different purposes $\rightarrow$ different best solutions

For different Data Structures, we need different operations to work with.

Data type:

- a data structure,
- with defined operations.

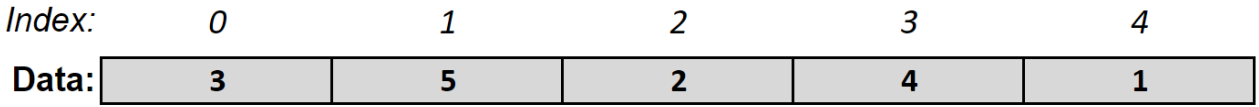
Data Type: Array

Array:

- The most common datatype
- Direct access to **data** through *indexing*.
- Notations:

$$A : \mathcal{T}[n]$$

  - A: Array
  - T: Type
  - n: The size of A
    - $A.length$
    - The maximum amount of data that can be stored in A.



Sorting problem

During the semester, we will learn various methods for this:
We need to sort a sequence of numbers into ascending order!
Example:

- input: (31, 41, 59, 26, 41, 58)
- correct output: (26, 31, 41, 41, 58, 59)

Which algorithm is best for a given application?
It depends on many factors, like:

- the number of items to be sorted,
- the extent to which the items are already somewhat sorted (like: 2,1,3,4,5)
- possible restrictions on the item values (like: only positive whole numbers)
- hardware configurations

Sorting problem

Why should we learn "simpler" straight methods instead of the "best" ones?

- Easier to understand at the beginning.
- Easier to implement into code.
- At small n (small amounts of data), they could be faster.

Ending

Thank you for your attention!